Task 02: How far to attaining knowledge?

Announced 02/Jun. Due: 14/Jun 10:00 PM

- Please write (only if true) the honor code. Explicitly state any source (person or thing) that you might have used, and prompts and links to LLM.
- Important: This tasks is solved in a team mode. If you choose to work individually, that's fine. However, the deadlines and load are considered with a team member in mind.
- Always provide explanations. Be verbose to explain your points. Your non-teammate should be able to understand you just by reading your text.
- Submission Guidelines. See `piazza` for standard static instructions.
- Your submission folder should look something like:

```
25v009_Task02/
├── answers.pdf
├── answers.tex
├── HW.sty
├── readme.txt
├── ReflectionEssay.pdf
├── images
│   ├── bicycle.jpg
│   ├── firstPersonView.png
│   ├── thirdPersonView.png
│   ├── otherImage1.png
│   └── otherImage2.png
├── task-body.tex
└── 02-task-header.tex
```

---

1 Overview

In this task, you will estimate real-world distances using a single image by – well – applying what you learnt in class. The task focuses on linearity, cross-ratio, and the practical use of object localization (using a deep learning tool) to infer distances from a single image. This topic is called single view metrology.

2 Background

You are provided with an image (see Fig. 1) of the way from the main building (MB) of IIT Bombay to the arch of knowledge (gyanam-paramam-dhyeyam) ("arch"). (As an aside, the arch encodes the motto of IITB).

Take a moment to digest the provided image. Observe

- the red line overlaid for reference,
- the cross road ("intersection") that has the palm tree on one side and a building on the other side

- a bicycle

Amongst the objects, look for the street lights towards the right. We refer to these as "poles" in this assignment. The poles are numbered starting from the camera to the Arch. We will casually refer to the red line as equivalent to MB. We are going to assume that the cross road (aka intersection) is a straight line passing through the base of the third pole parallel to the red line, and that the mb-to-arch line is perpendicular to the intersection line.

3 Who moved my bicycle?

3.1 Task

Consider the bicycle in Fig. 1. Report the real-world distance between the bicycle and the arch.

Figure 1: A bounding box prediction from YOLOv8.

Use a pretrained deep learning model to detect the bicycle. This is an object detection task. Please refer to tutorial here, and stick to **YOLO** models: https://docs.ultralytics.com/tasks/detect#models Also look at the COCO dataset https://cocodataset.org/#home to find out the class identifiers for bicycles and persons.

124.5m

3.2 Questions

You are expected to write answers inline in the student space provided. (see Section 5)

1. Use a tool from Google that tells you the real world distance from MB (the red line) to the arch. Use a number in metres and approximate to the nearest distance divisible by 10. 160 metres

using Google Maps.

2. Why can you NOT find the distance to the bicycle without making any assumption about the bicycle using the same tool? (Answer in one line.) We have to assume the bicycle is standing upright directly on the road normally and not floating in the air closer to MB because we lose depth information in projection.

3. Ensure that you draw a line in dark blue color (using an image annotation tool of your choice) that is used for obtaining the cross reference. Show this line and the corresponding image in your submission. How did you draw the line? Keep in mind that cross-ratio is a **1D** concept and requires 4 collinear points. Make sure these points lie on the dark blue color line above. Line was drawn by first drawing two lines along the footpaths to produce a vanishing point. Then the intended line was drawn from vanishing point towards the point on the bicycle. Further, it was extended to the MB red line.

4. Ensure that you annotate 4 points of interest as A', B', C', V' in sorted order of distance from the camera where B' corresponds to the biycle and V' corresponds to the vanishing point. What do A' and B' correspond to? We recommend using MS Paint for annotation and identifying pixel coordinates. A' corresponds to MB. B' corresponds to the bicycle.

5. Be sure to explain in detail your method (and code) for finding the bicycle. Does the choice of reference point within the bounding box from the deep learning model affect distance estimation accuracy? Quantify.

First, we wrote the code to detect objects using YOLO models and plot the bounding boxes on images.

Then, we get the vanishing point. We draw a straight line from vanishing point towards the MB with the bicycle in between. We note down the pixel values of the 4 points A'B'C'V'. From Google Maps, we know the —A'C'— = 160m. Assume —A'B'— = x m, then —B'C'— = 160-x m. By definition, —B'V'— = —C'V'— = inf. Also assuming that the line is perfectly vertical

line for simplicity, we note down pixel values (Y coord) of points: A'=3496, B'=2120, C'=1785, V'=1620 By equating the cross ratios in 2D pixel space and real world 3D space, we get: x = 160*1376*138/(500*1711) = 35.5m Therefore, distance between bicycle and arch is 124.5m

Yes the distance estimation changes based on the point in the bounding box. Assuming the bicycle is a thin plane in the real world, the line between any point inside the bounding box chosen to the actual point on the ground for the bicycle will be perpendicular to the correct line used for calculating cross ratio. Let that small error line distance be e. , Then: $\sqrt{x^2 + e^2}/160 = |A'B''| * 138/(1711 * |B''V'|)$

Where B" is the new bounding box point.

4 Your picture

In this section, take a picture of you and, alternately, your teammate (or some other person) similar to the one that has been provided in your locality. Identify two specific points of interest and tell us how far s/he is from the point of interest in the real world. We are expecting two pictures in your submission; one corresponding to the problem statement (often called as the first person view), and the second, a picture of you taking a picture of your teammate, i.e., a third person view. The second picture will show both the cameraman and the teammate (and ideally at the same time).

4.1 Tasks

1. Capture and include the images in your submission.
2. Detect the teammate using the same deep learning model.
3. Estimate the distance between one of the objects of interest and the team mate. Taking the point on team mate C' and the bottom of the vase A'. Pixel coords V' = (478, 510) C' = (610, 671) B' = (646, 711) A' = (736, 820) 60cm * inf / (x * inf) = A'B' * C'V' / A'C' * B'V' = 141.35 * 208.18 / (195.12 * 262) = 0.5755 x = 104.25cm

You are expected to write answers inline.

4.2 Question

1. As before, draw the line of interest (in blue color) marking all points of interest.
2. Explain how you got the real world distances. 60cm is the side of a tile.

3. Discuss practical challenges encountered while applying the projective geometry concepts to real-world images. Number all challenges and write independent challenges i.e. don't mix up the challenges. 1. Person is sitting: feet are hidden under a saree, so the true floor contact point is invisible and must be guessed. Bounding box point needs to be decided. 2. Vanishing point is too close: the the vanishing lines cross just 1.5 m away; a few pixels of error in V' causes large distance errors. 3. Objects not collinear: oil bottle and person's feet don't lie on the same line to V', requiring projection that adds approximation error. Finding collinear objects alongwith their real world coordinates is a hassle. 4. Shallow scene depth: the entire scene is around 2–3 m deep vs. 160 m in the IITB image. Small pixel errors is becoming large percentage errors in the result. 5. Background is cluttered: this reduces YOLO's detection confidence.

4. What programs did you write and why? We just wrote programs to detect and draw bounding boxes around the objects of interest by using YOLO models. Rest of the work was done in MSPaint calculator.

5 Submit

Write a description of your method and all relevant aspects of your experiments. Write it in line in the text under the relevant section. Your writing should be similar to a blog post so that anyone in the class can fully understand the pain/joy/content you experienced (feel free to take copious input from an LLM but do not substitute an LLM for learning).

The total length should be no less than 500 words and should normally exceed 800 words. Your ultimate aim in writing your answers to mention important aspect of your assignment for the task, with emphasis on the last task. This will include the methods employed listing all the software tools you employed, as well experimental aspects such as when you took the picture, how many pictures you took, whether the method works in dawn, morning, and dusk, and so on. Be creative.

Do not submit any large file such as model weights and ancillary files that we can easily get from the internet to reproduce.

First thing we had to was run the YOLO models locally. So we installed the required python package and wrote a script based on the given tutorials. However, this wasn't enough, the accuracy was still quite low and it wasn't able to detect the bicycle. This was because the YOLO library was automatically scaling the images down. Hence, we had to specify the size of the image, post which the models were able to see the correct image rather than the compressed image and hence were able to detect our objects of interest.

The code: "' from ultralytics import YOLO import cv2

Load a model model = YOLO("yolo26n.pt") load an official model

source = cv2.imread("images/bicycle.jpg")

Predict with the model results = model.predict(source, imgsz=(2068, 4032)) predict on an image

annotated = None Access the results for result in results: xywh = result.boxes.xywh center-x, center-y, width, height xywhn = result.boxes.xywhn normalized xyxy = result.boxes.xyxy top-left-x, top-left-y, bottom-right-x, bottom-right-y xyxyn = result.boxes.xyxyn normalized names = [result.names[cls.item()] for cls in result.boxes.cls.int()] class name of each box confs = result.boxes.conf confidence score of each box bicycle$_index = names.index('bicycle')result.boxes = result.boxes[bicycle_index : bicycle_index + 1]annotated = result.plot()$

cv2.imwrite('detected.jpg', annotated) "'

Next, we had to go find the real world distance between the arch and the Main Building red line. This was a little tricky as we first had to find the locations using Google Street view somehow and estimate approximately which points in the image coincided with the top-down view in google maps. Once we did that, it was straightforward to get the real world distance. Next, we had to draw the lines to get the collinear points. This was fairly straightforward because we just had to trace the fine line between the footpath and the road. By doing this for both left and right sides, we were able to extend the lines and get the vanishing point. Once we got the vanishing point, we had to choose a point in the bounding box which is collinear with the rest of the line on the road. Naturally, this had to be a point on/near the ground and on or on the same level as the bicycle tires. We drew a line from the vanishing point, through the bicycle point, extended all the way to the red line. By using MS Paint, we got the pixel coordinates for all the four points- Vanishing point, point near the third pole, the bicycle and the red line near the MB. Since cross ratio doesn't change regardless of any projective transformation, we equated the cross ratio from pixel world to the real world, using which we were able to get an equation that yielded the distance from the bicycle to the arch.